In today's digital era, a major concern affecting people, especially young students, is the rapid decline in their attention span and short-term memory retention. The widespread use of smartphones, social media, short-form content and multitasking environments has significantly altered the way individuals process information.

The follows studies provide a quantitative measure of the claims:

1. **Microsoft Attention Span Study (2015):**
    - Reported that the average human attention span has dropped from 12 seconds in 2000 to just 8 seconds in 2015.
    - This is shorter than the attention span of a Goldfish (9 seconds).
2. **Nielsen Norman Group Research (2023):**
    - Showed that typical users tend to focus on any single screen element for less than 2.8 seconds.
    - Young users (ages 13–24) switch their attention even faster, at about every 1.3 seconds.
3. **American Psychological Association (APA), 2021:**
    - Found that digital multitasking reduces learning efficiency by up to 40%, as the brain is forced to constantly re-orient itself.
4. **Harvard University Study (2020):**
    - People check their phones 96 times a day on average, i.e., once every 10 minutes; leading to continuous shifting of attention.
5. **National Institute of Mental Health (India), 2022:**
    - Increasing screen time among Indian students is strongly correlated with poor academic focus, weaker working memory and poor retention capabilities.

Impact On Students

- Students are exposed to fast-paced content (short-form videos, reels, etc.) that trains the brain to constantly expect external stimulation.
- This leads to:
    - Difficulty concentrating during studies
    - Poor retention and recall abilities
    - Increased dependency on external cues
    - Reduced mental stamina
    - Lower performance at tasks that require sustained attention (like exams)

Corrective Measures

To combat this decline, experts recommend games and exercises that strengthen:

- Focus and attention
- Short-term memory
- Information processing speed
- Cognitive flexibility
- Mindfulness and reaction time

Simple brain-training exercises have shown to improve cognitive response by 10–25% when practiced on a regular basis.

ABOUT THE PROJECT

Introduction

- The **Colour Memory Game** is a simple yet powerful cognitive skill building program. The objective of the program is to improve the player's **focus**, **attention span**, and **short-term memory**.

- In a world full of digital distractions, we are constantly losing our ability to concentrate for long durations. Our project presents an engaging and scientifically supported exercise for training the brain.

- The game is based on the **Stroop Effect**. It is psychological phenomenon in which the brain struggles to process conflicting information.

    - For example, when the word "**BLUE**" appears on the screen in red colour, the human brain is trained to immediately identify the text rather than to focus on identifying the colour in which the word is displayed. This exercise helps train the areas of the human brain that correspond to **memory**, **attention control**, and **mental processing speed**.

- In the **Color Memory Game,** the name of a colour is displayed. However, the font colour of the word will be different from the colour that the word corresponds to. The challenge for the player is not to identify the word, but to rather identify the font colour in which the word is displayed.

    - Example:
      If the screen shows:
      "**GREEN**"
      The correct answer is: BLUE

- The game becomes progressively more difficult as the time duration for which the font colour of the word is displayed decreases.

TECHNOLOGY USED

Python

Introduction



- Python is a **high-level, interpreted, general-purpose** programming language known for its readability and versatility.

- It was created by **Guido van Rossum** and was first released in **1991**.

- It allows beginners as well as advanced programmers to develop applications.

- Python is widely used in GUI development, AI, automation, data science, and much more.

How Python Powers Our Project

1. Implementing the Game Logic

Python has been used to create all the internal rules of the game, such as:

- decreasing the timer values
- updating the player's score
- checking if the player's input matches the displayed colour
- choosing colours and words at random
- deciding when the game should end

All decision-making in the program is performed through Python functions, loops, and conditional statements.

2. Managing the Graphical User Interface (GUI)

Using Python's built-in **tkinter module**, we created:

- the game window
- labels to display words and colours
- input boxes to take player input
- buttons
- timer progress bar
- score display

Python governs how these GUI elements are displayed, as well as how they behave and respond to the player's actions.

3. Handling User Interaction

Python has been employed to manage all user interactions, such as the following:

- requesting for the player's input
- reading the input
- screen updation on each user action

Management of user interactions allowed us to provide the user an opportunity to interact with the program.

4. Using Libraries and Modules

We imported the following libraries in python to support the functioning of the game:

- random - to randomly select colors and words
- tkinter - to build the basic graphical user interface (GUI)

- ttk - to employ modern themed widgets into the GUI

Python provides us with the means to access these features and makes it possible to combine these libraries into one working program.

5. Ensuring Program Reliability

Python's exception handling capabilities have been used to:

- check if the high score file exists
- creating the file if not yet created or deleted
- prevent the game from crashing abruptly

Python has helped us ensures a smooth user experience.

tkinter

Introduction



- Tkinter, short for **Tk Interface**, is the standard **Graphical User Interface (GUI)** library of Python**.**
- Tkinter was written by **Steen Lumholt** and **Guido van Rossum** (the inventor of the Python programming language) in 1991.
- It allows programmers to create desktop applications using simple Python code.
- Tkinter comes bundled with Python and does not require any external installation. It is available to the developer or the programmer by default.
- Tkinter provides a wide range of GUI components such as **windows, labels, buttons, text boxes, menus, frames**, and many more.
- These elements allow developers and programmer to create user-friendly and interactive applications as a result of its simplicity and ease of use.
- Tkinter is one of the most popular choices for beginners learning GUI development in Python.

How tkinter Powers Our Project

1. Creating the Main Game Window

- Tkinter's **Tk( ) function** is used to create the main application window where all elements of the game are displayed.

2. Displaying Game Information

Label widget from Tkinter has been used to display:

- Instructions to the player
- Current score
- Time remaining
- The word whose colour that the player must identify

Labels serve key functions as elements on the screen and help communicate all necessary information during game.

3. Taking User Input

- The **Entry widget** from Tkinter has been used to accept the player's typed responses.
- It allows players to type their response and makes the game interactive.

4. Creating Interactive Buttons

Tkinter's **Button widget** has been used for:

- Starting the game
- Restarting the game

These buttons allow the players to interaction with the program's different phases with ease and convenience.

5. Managing Keyboard Actions

- Tkinter is used to bind the 'Enter Key' to the execution of the function that checks the player's answer.

This makes the gameplay faster and furthermore interactive.

6. Managing Time Dependent Functions

Tkinter's **after( ) method** is used to create time-dependent effects such as:

- Countdown timer
- Delay before allowing the players to input their answers
- Timing messages such as "Ready" and "Go"

7. Updating Game Elements Dynamically

- Tkinter's **config( ) method** allows the program to update the text, the colour, the widget states and other features of an elements as the game progresses.

tkinter.ttk
Introduction



- ttk, short for **Themed tkinter**, is an extension of the Tkinter library that provides **modern, stylish**, and **visually improved** GUI widgets.
- While classic Tkinter widgets look simple and outdated, ttk offers a more polished and professional appearance that matches the appearance of modern operating systems.
- These widgets automatically adapt to the current system theme, making the interface look cleaner and more user-friendly.
- ttk also supports advanced styling options, allowing developers to create more attractive and responsive GUIs.

How tkinter.ttk Powers Our Project

1. Progress Bar

- The progress bar that compliments the timer in our game is implemented using **ttk.Progressbar( ).**
- This widget displays the time remaining as a moving bar. This provides the players a clear and more comprehensive visual countdown.

2. Difficulty Selection Menu

- We used **ttk.Combobox( )** to create a drop-down menu for selecting the difficulty level of the game.
- This allows players to choose from options like “Very Easy”, “Easy”, “Medium”, “Hard”, or “Very Hard” from a modern drop-box menu.

PROGRAM CODE

```
# ================================================
# CBSE CLASS 12 COMPUTER SCIENCE PROJECT
# PROGRAM: Color Memory Game
# ================================================

import tkinter as tk
import random
import csv
from tkinter import ttk

# ---------------- FILES ----------------
USERS_FILE = "users.csv"
LEADERBOARD_FILE = "leaderboard.csv"
HIGHSCORE_FILE = "highscores.csv"
current_user = None

# ---------------- GAME DATA ----------------
font_colors = ['Red', 'Blue', 'Green', 'Yellow', 'Orange',
               'Purple', 'Brown', 'Pink', 'Gray', 'White']

words = ['Red', 'Blue', 'Green', 'Yellow', 'Orange', 'Purple',
         'Brown', 'Pink', 'White', 'Gray']

difficulty_levels = {
    "Very Easy": 1.5,
    "Easy": 1.2,
    "Medium": 0.9,
    "Hard": 0.6,
    "Very Hard": 0.3
}

# ---------------- GAME VARIABLES ----------------
score = 0
high_score = 0
time_left = 30.0
current_color = ''
last_color = ''
timer_running = False
reveal_time = 1.0
countdown_interval = 100  # ms
```

```
# ================================================
# FILE INITIALIZATION
# ================================================

def initialize_user_file():
    try:
        open(USERS_FILE, "r").close()
    except FileNotFoundError:
        with open(USERS_FILE, "w", newline="") as f:
            writerobject = csv.writer(f)
            writerobject.writerow(["username", "password"])

def initialize_leaderboard():
    try:
        open(LEADERBOARD_FILE, "r").close()
    except FileNotFoundError:
        with open(LEADERBOARD_FILE, "w", newline="") as f:
            writerobject = csv.writer(f)
            writerobject.writerow(["username", "score"])
```

```
def initialize_highscores():
    try:
        open(HIGHSCORE_FILE, "r").close()
    except FileNotFoundError:
        with open(HIGHSCORE_FILE, "w", newline="") as f:
            writerobject = csv.writer(f)
            writerobject.writerow(["username", "high_score"])


# ================================================
# LOGIN & SIGNUP
# ================================================
def signup():

    username = username_entry.get().strip()
    password = password_entry.get().strip()

    if username == "" or password == "":
        message_label.config(text="All fields required", fg="red")
        return

    with open(USERS_FILE, "r") as f:
        for row in csv.reader(f):
            if row and row[0] == username:
                message_label.config(text="Username exists", fg="red")
                return

    with open(USERS_FILE, "a", newline="") as f:
        writerobject = csv.writer(f)
        writerobject.writerow([username, password])

    message_label.config(text="Signup successful. Login now.", fg="green")


def login():

    global current_user
    username = username_entry.get().strip()
    password = password_entry.get().strip()

    with open(USERS_FILE, "r") as f:
        readerobject = csv.reader(f)
        for row in readerobject:
            if row and row[0] == username and row[1] == password:
                current_user = username
                login_frame.destroy()
                show_game_ui()
                return

    message_label.config(text="Invalid login", fg="red")


# ================================================
# HIGH SCORE
# ================================================

def load_high_score():

    global high_score
    high score = 0
```

```
    with open(HIGHSCORE_FILE, "r") as f:
        readerobject = csv.reader(f)
        for row in readerobject:
            if row and row[0] == current_user:
                high_score = int(row[1])
                break
def save_high_score():

    rows = []
    updated = False

    with open(HIGHSCORE_FILE, "r") as f:
        readerobject = csv.reader(f)
        rows = list(readerobject)

    for row in rows:
        if row and row[0] == current_user:
            row[1] = str(high_score)
            updated = True

    if not updated:
        rows.append([current_user, high_score])

    with open(HIGHSCORE_FILE, "w", newline="") as f:
        writerobject = csv.writer(f)
        writerobject.writerows(rows)


# ================================================
# LEADERBOARD
# ================================================

def update_leaderboard(user, score):
    with open(LEADERBOARD_FILE, "a", newline="") as f:
        writerobject = csv.writer(f)
        writerobject.writerow([user, score])

def get_last_5():
    with open(LEADERBOARD_FILE, "r") as f:
        readerobject = csv.reader(f)
        data = list(readerobject)[1:]
    return data[-5:]


# ================================================
# GAME LOGIC
# ================================================
def countdown():
    global time_left, timer_running
    if timer_running and time_left > 0:
        time_left -= countdown_interval / 1000
        time_left_label.config(text=f"Time left: {int(time_left)}")
        timer_bar['value'] = time_left
        window.after(countdown_interval, countdown)
    elif time_left <= 0:
        timer_running = False
        end_game()
```

```
def on_start_click():

    global reveal_time, time_left, score
    score = 0
    score_label.config(text="Score: 0")
    level = difficulty_var.get()
    if level not in difficulty_levels:
        return

    reveal_time = difficulty_levels[level]
    time_left = 30.0
    load_high_score()

    start_button.pack_forget()
    level_menu.pack_forget()
    retry_button.pack_forget()

    word_label.config(text="Ready...")
    window.after(1000, lambda: word_label.config(text="Go!"))
    window.after(2000, start_round)

def start_round():
    global current_color, last_color
    entry.config(state='disabled')

    word = random.choice(words)
    possible_colors = [c for c in font_colors if c != word and c != last_color]
    current_color = random.choice(possible_colors)
    last_color = current_color

    word_label.config(text=word, fg=current_color)
    window.after(int(reveal_time * 1000), allow_input)

def allow_input():
    global timer_running
    word_label.config(fg="black")
    entry.config(state='normal')
    entry.focus()
    if not timer_running:
        timer_running = True
        countdown()

def next_color(event=None):
    global score
    if entry['state'] == 'disabled':
        return

    if entry.get().strip().lower() == current_color.lower():
        score += 1
        score_label.config(text=f"Score: {score}")

    entry.delete(0, tk.END)
    start_round()

def end_game():
    global high_score
    entry.config(state='disabled')

    if score > high_score:
        high_score = score
        save_high_score()
```

```
    update_leaderboard(current_user, score)

    text = "Leaderboard (Last 5 Games):\n"
    for i, row in enumerate(get_last_5(), 1):
        text += f"{i}. {row[0]} - {row[1]}\n"

    word_label.config(
        text=f"Game Over!\nScore: {score}\nHigh Score: {high_score}\n\n{text}",
        font=('Helvetica', 14))

    retry_button.pack(pady=10)

def retry_game():
    global score, timer_running, time_left
    score = 0
    time_left = 30.0
    timer_running = False

    score_label.config(text="Score: 0")
    time_left_label.config(text="Time left: 30")
    timer_bar['value'] = 30
    word_label.config(text="")
    entry.delete(0, tk.END)
    entry.config(state='disabled')

    start_button.pack(pady=10)
    level_menu.pack(pady=10)

# ================================================
# UI
# ================================================
def show_game_ui():
    window.title(f"Color Memory Game - {current_user}")

    instructions.pack(pady=10)
    score_label.pack()
    time_left_label.pack()
    timer_bar.pack(pady=5)
    word_label.pack(pady=20)
    entry.pack()
    level_menu.pack(pady=10)
    start_button.pack(pady=10)

# ================================================
# MAIN WINDOW
# ================================================
window = tk.Tk()
window.geometry("500x560")
window.title("Color Memory Game - Login")

initialize_user_file()
initialize_leaderboard()
initialize_highscores()

login_frame = tk.Frame(window)
login_frame.pack(pady=50)
```

```
tk.Label(login_frame,
         text="Login / Signup",
         font=("Helvetica", 18, "bold")).pack()
tk.Label(login_frame,
         text="Username").pack()
username_entry = tk.Entry(login_frame)
username_entry.pack()
tk.Label(login_frame,
         text="Password").pack()
password_entry = tk.Entry(login_frame,
                          show="*")
password_entry.pack()
message_label = tk.Label(login_frame)
message_label.pack()

tk.Button(login_frame,
          text="Login",
          command=login).pack(pady=5)
tk.Button(login_frame,
          text="Signup",
          command=signup).pack(pady=5)

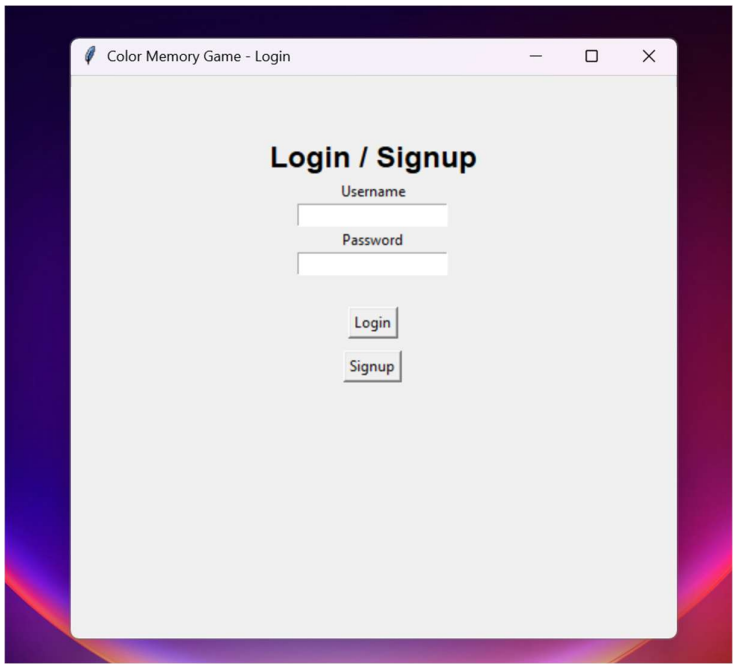
# Game widgets
instructions = tk.Label(window,
                        text="Type the COLOR of the word shown")
score_label = tk.Label(window,
                       text="Score: 0")
time_left_label = tk.Label(window,
                           text="Time left: 30")
timer_bar = ttk.Progressbar(window,
                            length=300,
                            maximum=30)
timer_bar['value'] = 30
word_label = tk.Label(window,
                      font=('Helvetica', 40))
entry = tk.Entry(window,
                 font=('Helvetica', 14),
                 state='disabled')

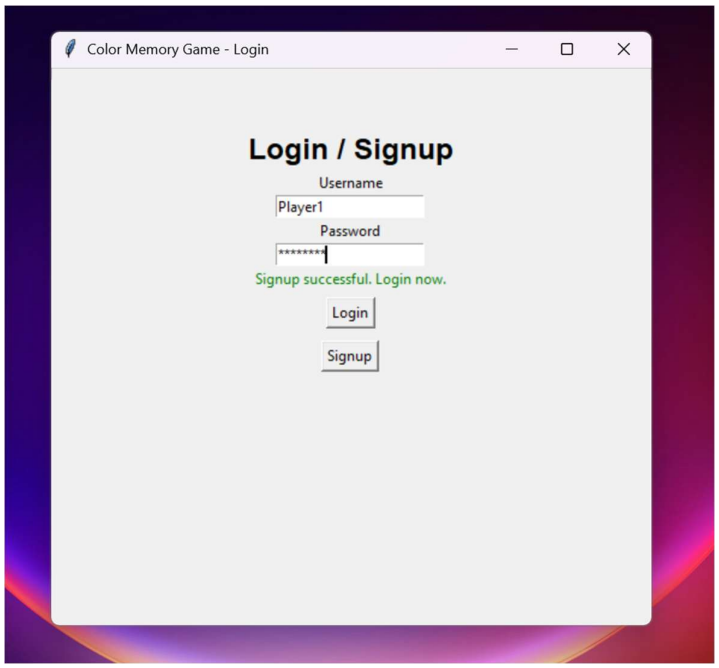
difficulty_var = tk.StringVar()
level_menu = ttk.Combobox(window,
                          textvariable=difficulty_var,
                          values=list(difficulty_levels.keys()),
                          state='readonly')
level_menu.set("Select Difficulty")

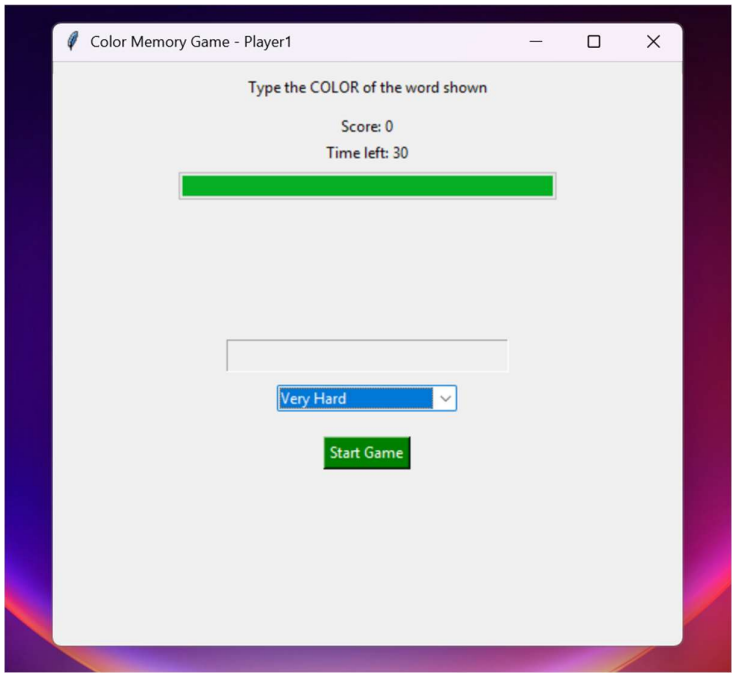
start_button = tk.Button(window,
                         text="Start Game",
                         bg="green",
                         fg="white",
                         command=on_start_click)
retry_button = tk.Button(window,
                         text="Play Again",
                         command=retry_game)

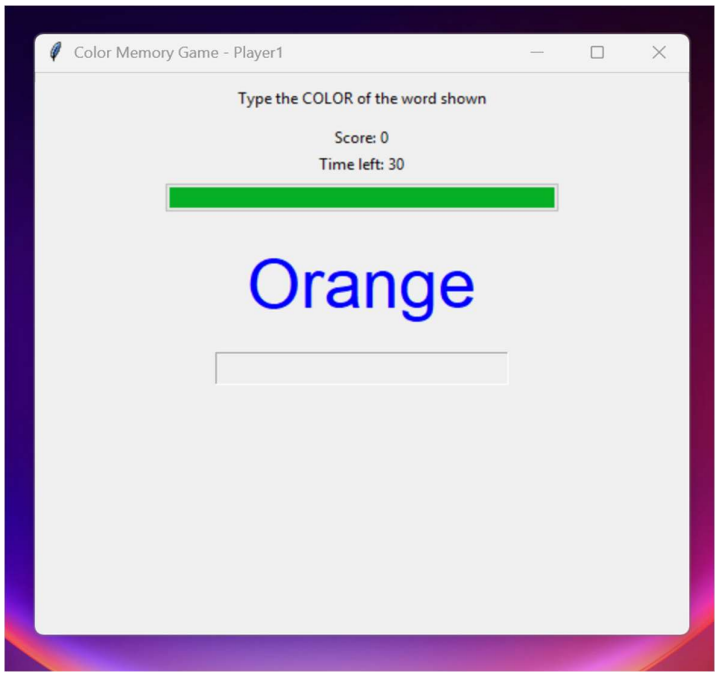
window.bind('<Return>', next_color)
window.mainloop()
```

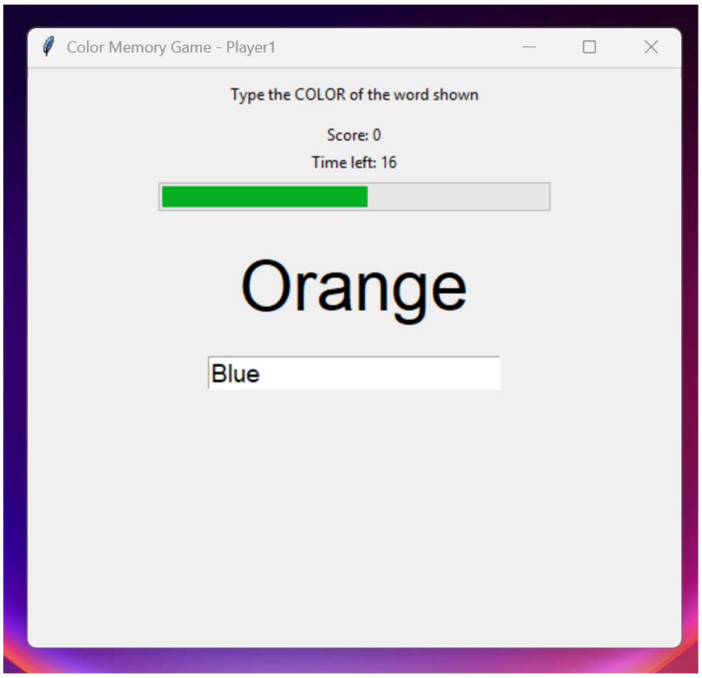
OUTPUT


Color Memory Game - Login
Login / Signup
Username
Password
Login
Signup

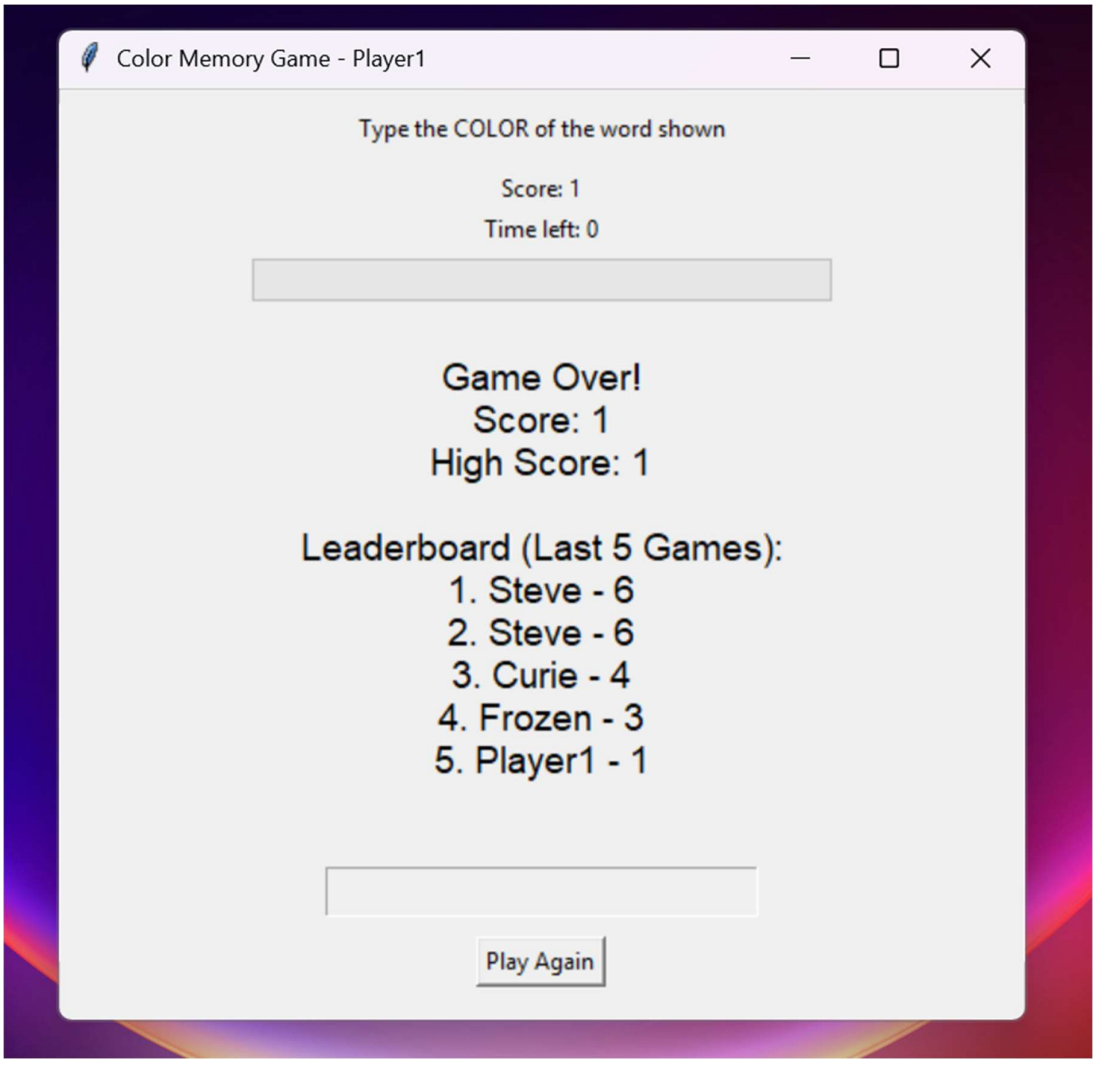

Color Memory Game - Login
Login / Signup
Username
Player1
Password
Signup successful. Login now.
Login
Signup

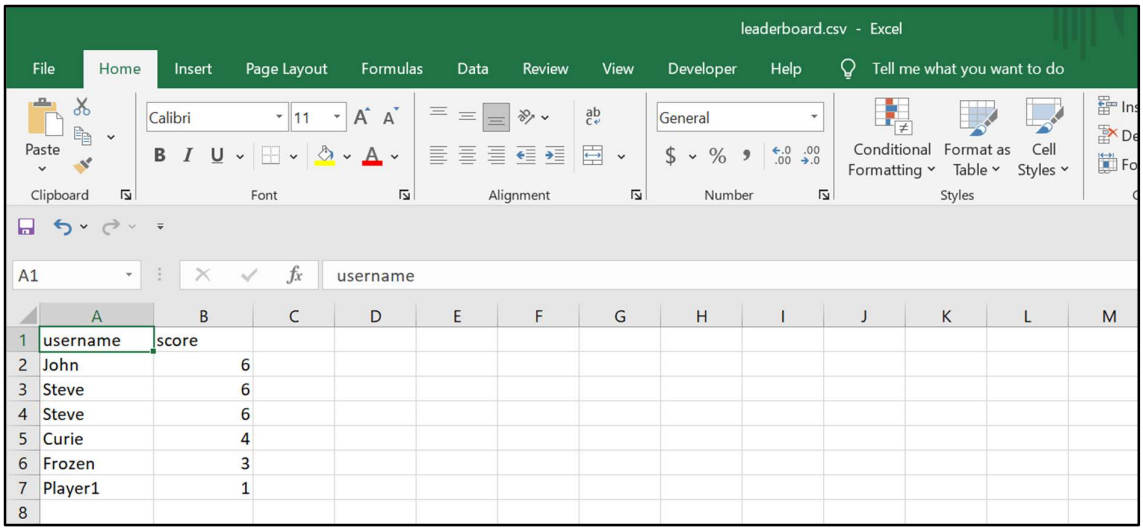

Color Memory Game - Player1
Type the COLOR of the word shown
Score: 0
Time left: 30
Very Hard
Start Game

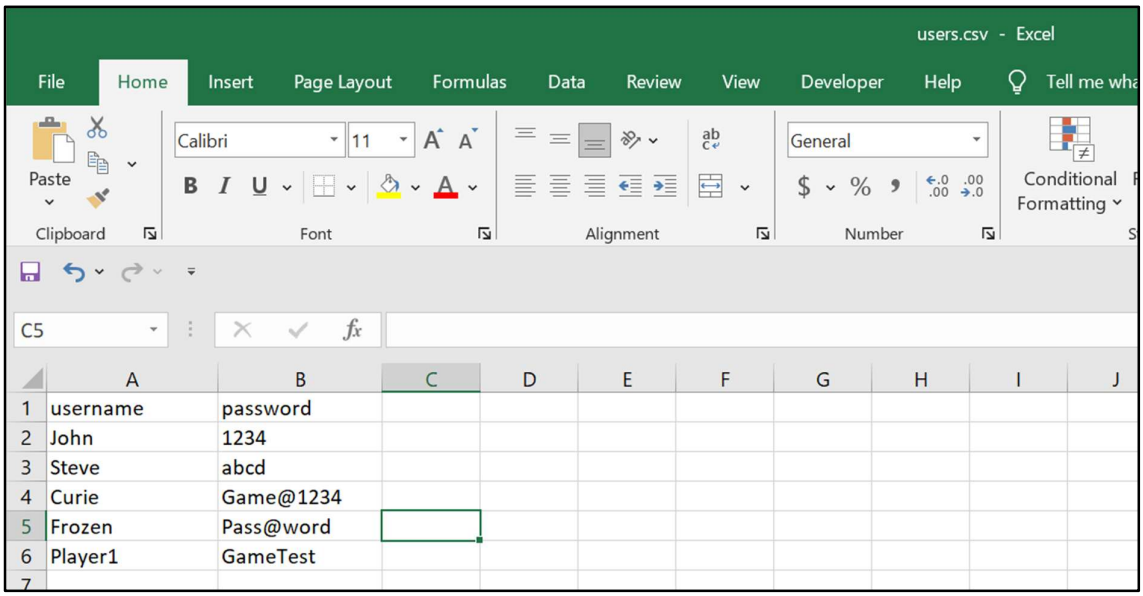

Color Memory Game - Player1
Type the COLOR of the word shown
Score: 0
Time left: 30
Orange

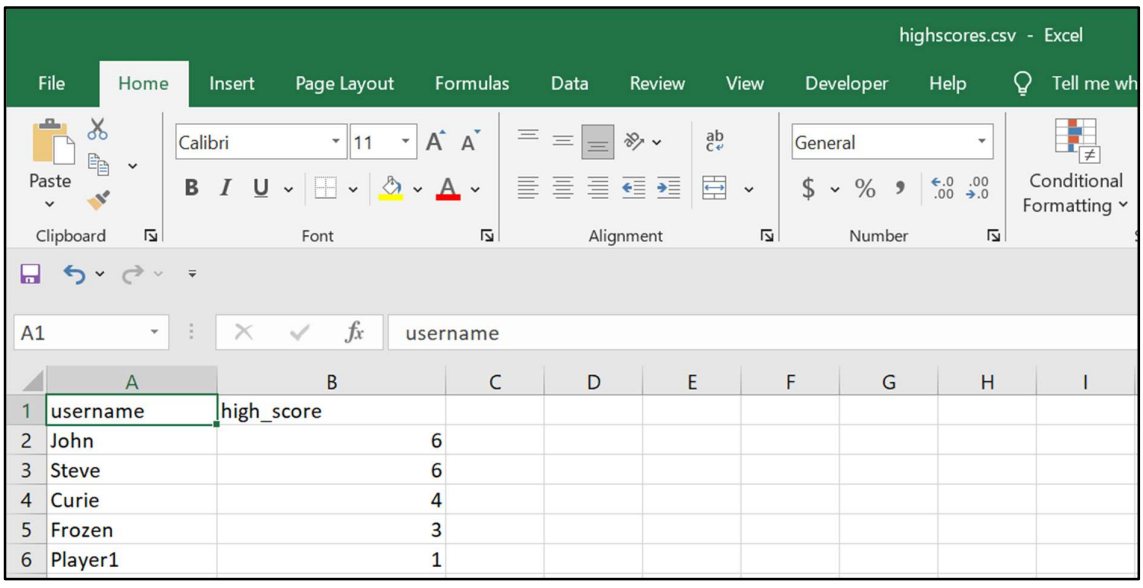

Color Memory Game - Player1
Type the COLOR of the word shown
Score: 0
Time left: 16
Orange
Blue


Color Memory Game - Player1
Type the COLOR of the word shown
Score: 1
Time left: 10
Pink


Color Memory Game - Player1
Type the COLOR of the word shown
Score: 1
Time left: 0
Game Over!
Score: 1
High Score: 1
Leaderboard (Last 5 Games):
1. Steve - 6
2. Steve - 6
3. Curie - 4
4. Frozen - 3
5. Player1 - 1
Play Again


leaderboard.csv - Excel
File Home Insert Page Layout Formulas Data Review View Developer Help Tell me what you want to do
Paste Clipboard Calibri 11 Font Alignment General Number Conditional Formatting Format as Table Cell Styles Styles
A1 username
A B C D E F G H I J K L M
1 username score
2 John 6
3 Steve 6
4 Steve 6
5 Curie 4
6 Frozen 3
7 Player1 1
8


users.csv - Excel
File Home Insert Page Layout Formulas Data Review View Developer Help Tell me wha
Paste Clipboard Calibri 11 Font Alignment General Number Conditional Formatting
C5
A B C D E F G H I J
1 username password
2 John 1234
3 Steve abcd
4 Curie Game@1234
5 Frozen Pass@word
6 Player1 GameTest
7


highscores.csv - Excel
File Home Insert Page Layout Formulas Data Review View Developer Help Tell me wh
Paste Clipboard Calibri 11 Font Alignment General Number Conditional Formatting
A1 username
A B C D E F G H I
1 username high_score
2 John 6
3 Steve 6
4 Curie 4
5 Frozen 3
6 Player1 1

BIBLIOGRAPHY

S.No.	Reference
1	Python Software Foundation. *Python Programming Language – Official Documentation*. Available at: https://www.python.org/doc/
2	Tkinter Documentation. *Tkinter — Python interface to Tcl/Tk*. Available at: https://docs.python.org/3/library/tkinter.html
3	Tkinter TTK Widgets. *Themed Tkinter Widgets (ttk) Documentation*. Available at: https://docs.python.org/3/library/tkinter.ttk.html
4	Sweigart, Al. *Automate the Boring Stuff with Python: Practical Programming for Total Beginners*. No Starch Press, 2015.
5	Lumholt, Steen, and Guido van Rossum. *Tkinter (Tk Interface) Library for Python*. Python.org, 1991.
6	Python Module Reference – *random*. Available at: https://docs.python.org/3/library/random.html
7	YouTube Video: *Python Tkinter GUI Tutorial - Build a Color Memory Game*. Available at: https://www.youtube.com/watch?v=TuLxsvK4svQ
8	W3Schools. *Python Tutorial*. Available at: https://www.w3schools.com/python/
9	NCERT. *Computer Science with Python Textbook for Class 12*. National Council of Educational Research and Training, India.